

Cahier des Charges Fonctionnel et Technique

Module : Gestion des Courses et Suivi du Transport Scolaire

SMARTBUS – Espace École

1. Contexte

Le projet **SMARTBUS** est une plateforme de gestion du transport intelligent.

Cette nouvelle fonctionnalité sera développée **uniquement dans le module "Espace École"**, destiné aux établissements scolaires utilisant SMARTBUS pour gérer le transport des élèves.

L'objectif est de permettre à une école de gérer l'ensemble de son transport scolaire depuis une interface unique, depuis la création des courses jusqu'à la validation automatique de la montée des élèves dans les cars grâce à la biométrie.

Le développement devra être intégré dans le projet existant **SMARTBUS_project**, sans modifier l'architecture actuelle.

2. Contexte Technique

Le projet est déjà composé de :

Backend

- NestJS 11
- TypeORM
- PostgreSQL
- BioTime 8.5 déjà intégré
- API REST

Application :

apps/super-app

Frontend

- React 19
- React Leaflet déjà installé
- Axios
- React Router

Application :

apps/school-web

Fonctionnalités déjà disponibles

Le projet possède déjà :

- la gestion des écoles
- la gestion des élèves
- la gestion des cars
- la biométrie BioTime
- la récupération des événements d'empreinte
- la récupération de la position GPS des cars

Cette nouvelle fonctionnalité devra exploiter ces composants existants.

3. Objectif Général

Développer un nouveau module dans SMARTBUS → Espace École permettant de :

- gérer les courses scolaires
- configurer les trajets des cars
- définir les points de récupération
- associer les élèves aux points
- suivre les cars
- valider automatiquement la montée des élèves grâce à leur empreinte digitale.

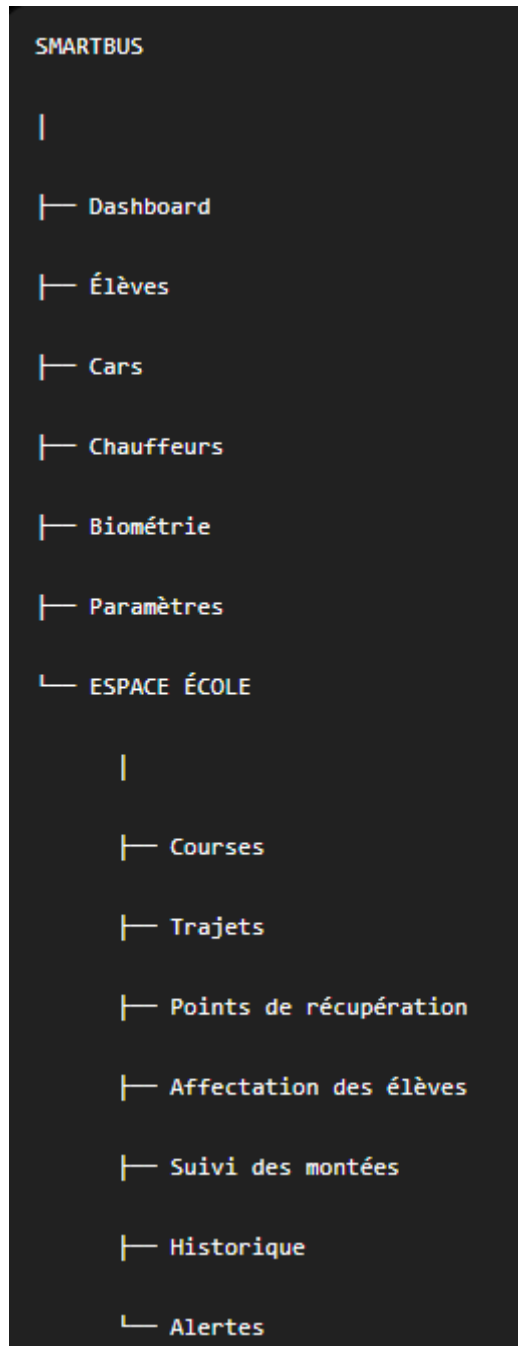
4. Objectifs Fonctionnels

L'application devra permettre de gérer entièrement le transport scolaire.

L'administrateur devra pouvoir :

- ✓ créer une course
- ✓ modifier une course
- ✓ supprimer une course
- ✓ affecter un car à une course
- ✓ dessiner le trajet de la course
- ✓ ajouter les points de récupération
- ✓ modifier les points
- ✓ supprimer les points
- ✓ définir leur ordre de passage
- ✓ associer les élèves
- ✓ consulter les élèves attendus
- ✓ suivre les validations de montée
- ✓ consulter les alertes

5. Architecture Fonctionnelle



6. Gestion des Courses

Une course représente une tournée scolaire.

Exemples :

- Aller Matin
- Retour Midi
- Retour Soir

Chaque course possède :

Nom
Description
Car
Chauffeur
École
Heure de départ
Heure d'arrivée
Jours d'exécution
Statut
Couleur sur la carte

Le système doit permettre :

- création
- modification
- suppression
- activation
- désactivation

7. Gestion des Trajets

Chaque course possède un trajet.

Le trajet est représenté par une **Polyline Leaflet.draw**.

L'utilisateur peut :

- dessiner le trajet
- modifier le trajet
- supprimer le trajet
- enregistrer le GeoJSON

Le système doit utiliser exclusivement :

Leaflet.draw

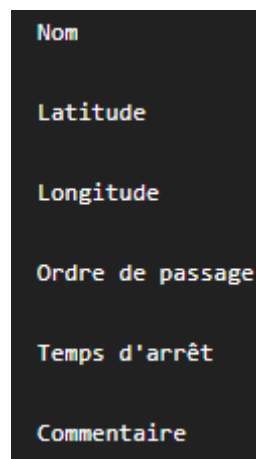
en s'inspirant de la démonstration officielle.

8. Gestion des Points de Récupération

Chaque trajet contient plusieurs points.

Chaque point est représenté par un Marker.

Informations :



Nom
Latitude
Longitude
Ordre de passage
Temps d'arrêt
Commentaire

Fonctionnalités :

Ajouter

Modifier

Supprimer

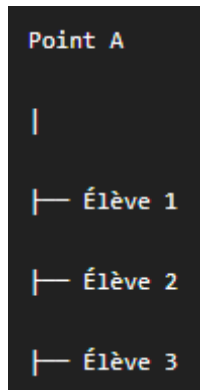
Déplacer

Réordonner

9. Affectation des Élèves

Chaque point peut recevoir plusieurs élèves.

Exemple



L'utilisateur doit pouvoir :

rechercher un élève

sélection multiple

supprimer un élève

voir les élèves déjà affectés

10. Carte Interactive

La carte devra permettre :

✓ dessiner une course

✓ modifier une course

✓ supprimer une course

✓ créer un point

✓ déplacer un point

✓ modifier un point

✓ afficher les élèves

✓ afficher le car

✓ afficher le trajet

11. Sauvegarde

Toutes les modifications devront être sauvegardées au format GeoJSON.

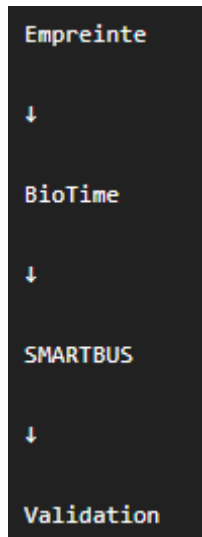
Exemple :

```
</> JSON

{
  "courseId": 1,
  "carId": 5,
  "geometry": {},
  "points": [
    {
      "id": 2,
      "ordre": 1,
      "latitude": 5.372,
      "longitude": -4.008,
      "enfantsAttendus": [15, 28]
    }
  ]
}
```

12. Validation Biométrique

Lorsqu'un enfant badge :

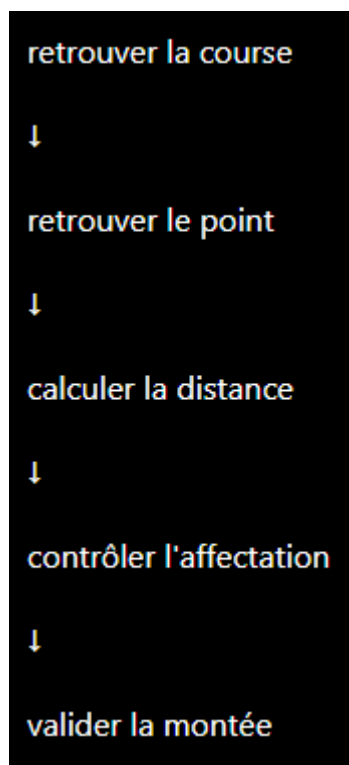


Le backend doit :

récupérer

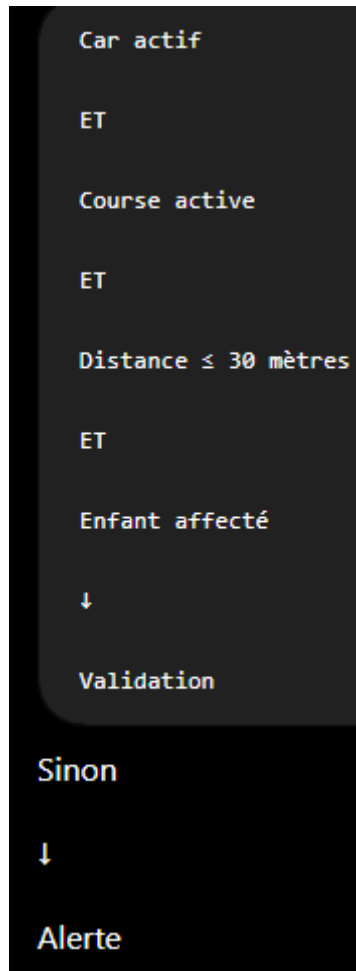
- id_enfant
- heure
- terminal
- car
- GPS du car

Puis :



13. Algorithme de Validation

Conditions :



14. Alertes

Le système devra générer des alertes lorsque :

- mauvais car
- mauvais arrêt
- course inactive
- enfant non affecté
- GPS hors zone
- badge en dehors de l'horaire
- double validation

15. Historique

Chaque montée doit être enregistrée.

Informations :

Élève
Course
Car
Point
Date
Heure
Distance GPS
Statut
Validation

16. Tableau de Bord

Afficher :

Nombre de courses

Nombre de cars actifs

Nombre d'élèves transportés

Montées validées

Montées refusées

Alertes

Courses en cours

17. API REST

Créer les endpoints :

```
GET /courses
```

```
POST /courses
```

```
PUT /courses/:id
```

```
DELETE /courses/:id
```

```
GET /courses/:id
```

```
POST /trajets
```

```
PUT /trajets/:id
```

```
DELETE /trajets/:id
```

```
POST /points
```

```
PUT /points/:id
```

```
DELETE /points/:id
```

```
POST /points/:id/enfants
```

```
DELETE /points/:id/enfants/:id
```

```
GET /montées
```

```
POST /validation
```

```
GET /alertes
```

18. Contraintes Techniques

- Frontend : React 19
- Backend : NestJS 11
- ORM : TypeORM
- Base de données : PostgreSQL (avec PostGIS si disponible pour les calculs géographiques)
- Cartographie : React Leaflet + Leaflet.draw (obligatoire)
- Communication : Axios + API REST
- Validation biométrique : réutiliser le service biotime.service.ts existant, sans le réécrire.

19. Livrables

Le développement devra inclure :

- Les nouvelles entités TypeORM
- Les migrations de base de données
- Les DTOs
- Les contrôleurs REST
- Les services NestJS
- Les composants React
- L'éditeur cartographique avec Leaflet.draw
- Les hooks React
- Les appels Axios
- Les calculs de distance GPS
- La logique de validation biométrique
- Les écrans de suivi des montées
- Les écrans d'historique et d'alertes
- Les tests unitaires des services critiques
- Une documentation technique et fonctionnelle décrivant l'architecture, les flux de données et les règles métier.

Recommandation d'architecture

Pour garantir la maintenabilité, je recommande de distinguer clairement les notions métier suivantes :

- **Car** : le véhicule.
- **Course** : la tournée planifiée (avec horaires, véhicule, école, statut).
- **Trajet** : le parcours géographique de la course (Polyline).
- **Point de récupération** : un arrêt appartenant à un trajet.
- **Affectation** : la relation entre un élève et un point de récupération.
- **Montée** : l'événement validé lorsqu'un élève monte dans le car.
- **Alerte** : un événement anormal détecté lors de la validation.

Cette séparation rendra le module plus évolutif (gestion de plusieurs écoles, modifications de trajets, historiques, rapports et analyses) et facilitera son intégration dans l'architecture existante de **SMARTBUS – Espace École**.